

**LAPORAN PRAKTIKUM
IMPLEMENTASI PERANGKAT LUNAK**

UAS

DISUSUN OLEH :

Ricky Gunaldo 2350081028



**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI
TAHUN 2025**

DAFTAR ISI

DAFTAR ISI.....	i
DAFTAR GAMBAR	ii
BAB I. HASIL PRAKTIKUM.....	3
I.1 Nomor 1	3
I.1.A. Source Code BidangDatar.java.....	3
I.1.B. Analisa	6
I.2 Nomor 2	6
I.2.A. Source Code BidangDatar.java (Interface)	6
I.2.B. Source Code Persegi.java.....	7
I.2.C. Source Code PersegiPanjang.java.....	8
I.2.D. Source Code Lingkaran.java.....	9
I.2.E. Analisa.....	10
I.3 Nomor 3	11
I.3.A. Source Code BidangDatarApps.java	11
I.3.B. Screenshot	15
I.3.C. Analisa	15
I.4 Nomor 4	16
I.4.A. Source Code Segitiga.java	16
I.4.B. Source Code BidangDatarApps.java (Update Segitiga)	17
I.4.C. Screenshot	22
I.4.D. Analisa	23

DAFTAR GAMBAR

<i>Gambar I.1 Output BidangDatarApps.java</i>	<i>15</i>
<i>Gambar I.2 Output BidangDatarApps.java (Menghitung Keliling Segitiga).....</i>	<i>22</i>
<i>Gambar I.3 Output BidangDatarApps.java (Menghitung Luas Segitiga)</i>	<i>23</i>

BAB I. HASIL PRAKTIKUM

I.1 Nomor 1

I.1.A. Source Code BidangDatar.java

```
package Sebelum;

public class BidangDatar {

    private double panjang;

    private double lebar;

    private double sisi;

    private double jariJari;

    private double attribute;

    private final double PI = 3.14;

    public double getPanjang() {

        return panjang;

    }

    public void setPanjang(double panjang) {

        this.panjang = panjang;

    }

    public double getLebar() {

        return lebar;

    }

}
```

```
public void setLebar(double lebar) {  
    this.lebar = lebar;  
}  
  
public double getSisi() {  
    return sisi;  
}  
  
public void setSisi(double sisi) {  
    this.sisi = sisi;  
}  
  
public double getJariJari() {  
    return jariJari;  
}  
  
public void setJariJari(double jariJari) {  
    this.jariJari = jariJari;  
}  
  
public double getAttribute() {  
    return attribute;  
}  
  
public void setAttribute(double attribute) {  
    this.attribute = attribute;  
}
```

```
}

public double kelilingPersegi() {
    return 4 * sisi;
}

public double luasPersegi() {
    return sisi * sisi;
}

public double kelilingPersegiPanjang() {
    return (2 * panjang) + (2 * lebar);
}

public double luasPersegiPanjang() {
    return panjang * lebar;
}

public double kelilingLingkaran() {
    return 2 * PI * jariJari;
}

public double luasLingkaran() {
    return PI * jariJari * jariJari;
}
}
```

I.1.B. Analisa

Implementasi kelas BidangDatar menunjukkan adanya pelanggaran terhadap prinsip Single Responsibility Principle (SRP) dari standar SOLID. Dalam kode, satu kelas tunggal dipaksa untuk menangani berbagai tanggung jawab yang berbeda, mulai dari menyimpan atribut untuk persegi, persegi panjang, hingga lingkaran secara bersamaan dalam satu ruang lingkup. Struktur kode seperti ini meningkatkan kompleksitas kognitif bagi pengembang dan memperbesar risiko terjadinya efek samping yang tidak diinginkan saat melakukan perubahan pada salah satu rumus bangun datar.

Kode ini juga melanggar prinsip Open/Closed Principle (OCP) karena strukturnya yang kaku dan tidak mudah dikembangkan. Jika di masa mendatang terdapat kebutuhan untuk menambahkan jenis bangun datar baru, pengembang terpaksa harus melakukan modifikasi langsung pada kelas BidangDatar yang sudah ada daripada melakukan ekstensi. Hal ini sangat tidak efisien secara sistematis karena setiap penambahan fitur baru akan menuntut pengujian ulang menyeluruh terhadap seluruh fungsi kelas untuk memastikan tidak ada logika yang rusak. Ketergantungan yang tinggi antar variabel di dalam kelas ini menciptakan hubungan yang terlalu erat (high coupling), yang pada akhirnya menghambat skalabilitas aplikasi dan mempersulit proses implementasi pengujian unit yang terisolasi.

I.2 Nomor 2

I.2.A. Source Code BidangDatar.java (Interface)

```
package Sesudah;

public interface BidangDatar {

    double hitungLuas();

    double hitungKeliling();
```

```
}
```

I.2.B. Source Code Persegi.java

```
package Sesudah;

public class Persegi implements BidangDatar {
    private double sisi;

    public Persegi(double sisi) {
        this.sisi = sisi;
    }

    public double getSisi() {
        return sisi;
    }

    public void setSisi(double sisi) {
        this.sisi = sisi;
    }

    @Override
    public double hitungKeliling() {
        return 4 * sisi;
    }

    @Override
    public double hitungLuas() {
```

```
        return sisi * sisi;
    }
}
```

I.2.C. Source Code PersegiPanjang.java

```
package Sesudah;

public class PersegiPanjang implements
BidangDatar {
    private double panjang;
    private double lebar;

    public PersegiPanjang(double panjang, double
lebar) {
        this.panjang = panjang;
        this.lebar = lebar;
    }

    public double getPanjang() {
        return panjang;
    }

    public void setPanjang(double panjang) {
        this.panjang = panjang;
    }

    public double getLebar() {
        return lebar;
    }
}
```

```

    }

    public void setLebar(double lebar) {
        this.lebar = lebar;
    }

    @Override
    public double hitungKeliling() {
        return (2 * panjang) + (2 * lebar);
    }

    @Override
    public double hitungLuas() {
        return panjang * lebar;
    }
}

```

I.2.D. Source Code Lingkaran.java

```

package Sesudah;

public class Lingkaran implements BidangDatar {
    private double jariJari;
    private final double PI = 3.14;

    public Lingkaran(double jariJari) {
        this.jariJari = jariJari;
    }
}

```

```

        public double getJariJari() {
            return jariJari;
        }

        public void setJariJari(double jariJari) {
            this.jariJari = jariJari;
        }

        @Override
        public double hitungKeliling() {
            return 2 * PI * jariJari;
        }

        @Override
        public double hitungLuas() {
            return PI * jariJari * jariJari;
        }
    }
}

```

I.2.E. Analisa

Perubahan pada kode ini memecah ketergantungan yang kaku pada kelas tunggal dan mendistribusikan tanggung jawab ke masing-masing kelas spesifik seperti Persegi, PersegiPanjang, dan Lingkaran sesuai dengan prinsip Single Responsibility Principle (SRP). Setiap kelas kini hanya menyimpan atribut yang benar-benar relevan dengan entitasnya, seperti atribut sisi yang hanya ada pada kelas Persegi, atau jariJari pada kelas Lingkaran, sehingga menghilangkan redundansi data dan variabel tidak terpakai yang sebelumnya muncul pada kode

versi lama. Penggunaan interface ini memungkinkan abstraksi yang kuat, di mana logika perhitungan luas dan keliling didefinisikan secara unik melalui metode `hitungLuas()` dan `hitungKeliling()` yang wajib diimplementasikan oleh setiap kelas turunan.

Kode ini juga telah menerapkan prinsip Open/Closed Principle (OCP) yang memungkinkan sistem untuk menerima penambahan jenis bangun datar baru di masa depan tanpa harus memodifikasi kode yang sudah ada, cukup dengan membuat kelas baru yang mengimplementasikan antarmuka `BidangDatar`. Hal ini menciptakan logika program yang lebih fleksibel dan relevan dengan standar pengembangan perangkat lunak modern yang menuntut pemeliharaan efisien serta pengujian yang terisolasi. Selain itu, penggunaan anotasi `@Override` pada setiap metode perhitungan memastikan penerapan polimorfisme berjalan dengan benar, sehingga setiap objek dapat memberikan hasil yang akurat sesuai rumus geometrisnya masing-masing namun tetap berada dalam satu payung tipe data yang konsisten. Arsitektur berbasis SOLID ini tidak hanya meningkatkan keterbacaan kode secara keseluruhan, tetapi juga memperkuat integritas sistem dalam menghadapi kompleksitas aplikasi yang terus berkembang secara logis dan terorganisir.

I.3 Nomor 3

I.3.A. Source Code BidangDatarApps.java

```
package Sesudah;

import java.util.Scanner;

public class BidangDatarApps {
    public static void main(String[] args) {
```

```

        Scanner scanner = new
Scanner(System.in);

        System.out.println("Pilih Bidang
Datar");

        System.out.println("1. Lingkaran");
        System.out.println("2. Persegi");
        System.out.println("3. Persegi
Panjang");

        System.out.print("Pilih: ");
        int pilihan = scanner.nextInt();

        if (pilihan == 1) {
            System.out.println("Masukan Jari-
jari Lingkaran");

            double jariJari =
scanner.nextDouble();

            Lingkaran lingkaran = new
Lingkaran(jariJari);

            System.out.println("Pilih Hitung
Keliling atau Luas");

            System.out.println("1. Keliling");
            System.out.println("2. Luas");
            System.out.print("Pilih: ");
            int menuHitung = scanner.nextInt();

```

```

        if (menuHitung == 1) {

            System.out.println("Keliling
Lingkaran          adalah          "          +
lingkaran.hitungKeliling());

        } else if (menuHitung == 2) {

            System.out.println("Luas
Lingkaran adalah " + lingkaran.hitungLuas());

        }

    } else if (pilihan == 2) {

        System.out.println("Masukan      Sisi
Persegi");

        double sisi = scanner.nextDouble();

        Persegi persegi = new Persegi(sisi);

        System.out.println("Pilih      Hitung
Keliling atau Luas");

        System.out.println("1. Keliling");
        System.out.println("2. Luas");
        System.out.print("Pilih: ");

        int menuHitung = scanner.nextInt();

        if (menuHitung == 1) {

            System.out.println("Keliling
Persegi adalah " + persegi.hitungKeliling());

        } else if (menuHitung == 2) {

            System.out.println("Luas
Persegi adalah " + persegi.hitungLuas());

```

```

    }

    } else if (pilihan == 3) {

        System.out.println("Masukan
Panjang");

        double          panjang          =
scanner.nextDouble();

        System.out.println("Masukan
Lebar");

        double lebar = scanner.nextDouble();

        PersegiPanjang    pp          =          new
PersegiPanjang(panjang, lebar);

        System.out.println("Pilih          Hitung
Keliling atau Luas");

        System.out.println("1. Keliling");
        System.out.println("2. Luas");
        System.out.print("Pilih: ");
        int menuHitung = scanner.nextInt();

        if (menuHitung == 1) {

            System.out.println("Keliling
Persegi Panjang adalah " + pp.hitungKeliling());

        } else if (menuHitung == 2) {

            System.out.println("Luas
Persegi Panjang adalah " + pp.hitungLuas());

        }

    } else {

```

```

        System.out.println("Pilihan
tidak valid.");
    }

    scanner.close();
}
}

```

I.3.B. Screenshot

```

Run  BidangDatarApps x
C:\Program Files\Java\jdk-1.8\bin\java.exe" ...
Pilih Bidang Datar
1. Lingkaran
2. Persegi
3. Persegi Panjang
Pilih: 2
Masukan Sisi Persegi
4
Pilih Hitung Keliling atau Luas
1. Keliling
2. Luas
Pilih: 2
Luas Persegi adalah 16.0
Process finished with exit code 0

```

Gambar I.1 Output BidangDatarApps.java

I.3.C. Analisa

Kode ini berfungsi sebagai pintu masuk utama aplikasi yang menangani interaksi langsung dengan pengguna melalui menu input di command line. Peran utamanya telah bergeser dari pengelola logika matematika menjadi pengelola alur program (flow controller), yang bertugas memvalidasi pilihan pengguna dan menginstansiasi objek

yang tepat. Dengan menggunakan struktur if-else, program dapat memberikan pengalaman interaktif sesuai dengan spesifikasi input yang diminta.

Penerapan polimorfisme di dalam file ini terlihat saat variabel tipe BidangDatar digunakan untuk menampung berbagai objek bangun datar yang berbeda. Hal ini membuktikan bahwa class driver tidak perlu mengetahui detail teknis cara menghitung luas atau keliling, ia hanya perlu memanggil metode yang sudah didefinisikan dalam interface. Pendekatan ini membuat kode driver lebih sederhana dan tidak mudah error meskipun jenis bidang datar terus bertambah.

I.4 Nomor 4

I.4.A. Source Code Segitiga.java

```
package Sesudah;

public class Segitiga implements BidangDatar {
    private double alas;
    private double tinggi;
    private double sisiMiring;

    public Segitiga(double alas, double tinggi)
    {
        this.alas = alas;
        this.tinggi = tinggi;
        this.sisiMiring = Math.sqrt((alas *
    alas) + (tinggi * tinggi));
    }

    public double getAlas() { return alas; }
```

```

        public void setAlas(double alas) { this.alas
= alas; }

        public double getTinggi() { return tinggi; }

        public void setTinggi(double tinggi) {
this.tinggi = tinggi; }

        public double getSisiMiring() { return
sisiMiring; }

        @Override

        public double hitungKeliling() {

            return alas + tinggi + sisiMiring;

        }

        @Override

        public double hitungLuas() {

            return 0.5 * alas * tinggi;

        }

    }

```

I.4.B. Source Code BidangDatarApps.java (Update Segitiga)

```

package Sesudah;

import java.util.Scanner;

public class BidangDatarApps {

    public static void main(String[] args) {

```

```

        Scanner scanner = new
Scanner(System.in);

        System.out.println("Pilih Bidang
Datar");

        System.out.println("1. Lingkaran");
        System.out.println("2. Persegi");
        System.out.println("3. Persegi
Panjang");

        System.out.println("4. Segitiga Siku");

        System.out.print("Pilih: ");
        int pilihan = scanner.nextInt();

        if (pilihan == 1) {
            System.out.println("Masukan Jari-
jari Lingkaran");

            double jariJari =
scanner.nextDouble();

            Lingkaran lingkaran = new
Lingkaran(jariJari);

            System.out.println("Pilih Hitung
Keliling atau Luas");

            System.out.println("1. Keliling");
            System.out.println("2. Luas");
            System.out.print("Pilih: ");
            int menuHitung = scanner.nextInt();

```

```

        if (menuHitung == 1) {
            System.out.println("Keliling
Lingkaran          adalah          "          +
lingkaran.hitungKeliling());
        } else if (menuHitung == 2) {
            System.out.println("Luas
Lingkaran adalah " + lingkaran.hitungLuas());
        }

    } else if (pilihan == 2) {
        System.out.println("Masukan      Sisi
Persegi");
        double sisi = scanner.nextDouble();

        Persegi persegi = new Persegi(sisi);

        System.out.println("Pilih      Hitung
Keliling atau Luas");
        System.out.println("1. Keliling");
        System.out.println("2. Luas");
        System.out.print("Pilih: ");
        int menuHitung = scanner.nextInt();

        if (menuHitung == 1) {
            System.out.println("Keliling
Persegi adalah " + persegi.hitungKeliling());
        } else if (menuHitung == 2) {

```

```

        System.out.println("Luas
Persegi adalah " + persegi.hitungLuas());

    }

    } else if (pilihan == 3) {

        System.out.println("Masukan
Panjang");

        double                panjang                =
scanner.nextDouble();

        System.out.println("Masukan
Lebar");

        double lebar = scanner.nextDouble();

        PersegiPanjang        pp        =        new
PersegiPanjang(panjang, lebar);

        System.out.println("Pilih        Hitung
Keliling atau Luas");

        System.out.println("1. Keliling");
        System.out.println("2. Luas");
        System.out.print("Pilih: ");
        int menuHitung = scanner.nextInt();

        if (menuHitung == 1) {

            System.out.println("Keliling
Persegi Panjang adalah " + pp.hitungKeliling());

        } else if (menuHitung == 2) {

            System.out.println("Luas
Persegi Panjang adalah " + pp.hitungLuas());

```

```

        }

        } else if (pilihan == 4) {

            System.out.println("Masukan      Alas
Segitiga");

            double alas = scanner.nextDouble();

            System.out.println("Masukan      Tinggi
Segitiga");

            double            tinggi            =
scanner.nextDouble();

            Segitiga      segitiga      =      new
Segitiga(alas, tinggi);

            System.out.println("Pilih      Hitung
Keliling atau Luas");

            System.out.println("1. Keliling");
            System.out.println("2. Luas");

            System.out.print("Pilih: ");

            int menuHitung = scanner.nextInt();

            if (menuHitung == 1) {

                System.out.println("Keliling
Segitiga adalah " + segitiga.hitungKeliling());

            } else if (menuHitung == 2) {

                System.out.println("Luas
Segitiga adalah " + segitiga.hitungLuas());

            }

        } else {

```

```

        System.out.println("Pilihan
tidak valid.");

    }

    scanner.close();

}

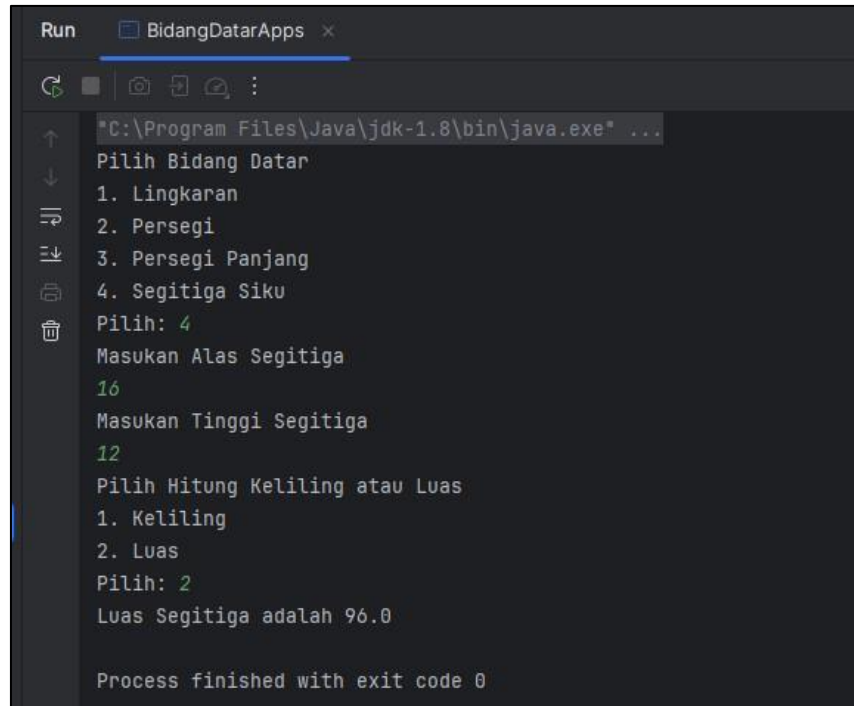
}

```

I.4.C. Screenshot

The screenshot shows a Java application window titled "BidangDatarApps". The application prompts the user to "Pilih Bidang Datar" (Choose Flat Shape) and lists four options: 1. Lingkaran (Circle), 2. Persegi (Square), 3. Persegi Panjang (Rectangle), and 4. Segitiga Siku (Right Triangle). The user selects option 4. The application then prompts for the base of the triangle ("Masukan Alas Segitiga") and the user enters 16. Next, it prompts for the height ("Masukan Tinggi Segitiga") and the user enters 12. The application then prompts the user to "Pilih Hitung Keliling atau Luas" (Choose Calculate Perimeter or Area) and lists two options: 1. Keliling (Perimeter) and 2. Luas (Area). The user selects option 1. The application then displays the result: "Keliling Segitiga adalah 48.0" (The perimeter of the triangle is 48.0). The window also shows the command prompt path "C:\Program Files\Java\jdk-1.8\bin\java.exe" and the message "Process finished with exit code 0".

Gambar 1.2 Output BidangDatarApps.java (Menghitung Keliling Segitiga)



```
Run  BidangDatarApps x
Pilih Bidang Datar
1. Lingkaran
2. Persegi
3. Persegi Panjang
4. Segitiga Siku
Pilih: 4
Masukan Alas Segitiga
16
Masukan Tinggi Segitiga
12
Pilih Hitung Keliling atau Luas
1. Keliling
2. Luas
Pilih: 2
Luas Segitiga adalah 96.0

Process finished with exit code 0
```

Gambar 1.3 Output BidangDatarApps.java (Menghitung Luas Segitiga)

I.4.D. Analisa

Pada kode ini membuktikan fleksibilitas arsitektur perangkat lunak yang menggunakan prinsip Open/Closed Principle (OCP). Dengan adanya kebutuhan untuk menambahkan bangun datar baru berupa Segitiga, pengembang tidak perlu merombak logika inti pada antarmuka BidangDatar, melainkan cukup membuat kelas baru yang mengimplementasikan kontrak tersebut. Kelas Segitiga ini secara logis mengenkapsulasi atribut spesifik seperti alas, tinggi, dan sisiMiring, serta melakukan kalkulasi internal terhadap sisi miring menggunakan teorema Pythagoras melalui fungsi `Math.sqrt` dalam konstruktornya. Hal ini menunjukkan konsistensi dalam penerapan Single Responsibility Principle (SRP), di mana setiap kelas memiliki otoritas penuh atas logika matematisnya sendiri, seperti metode `hitungKeliling()` yang menjumlahkan ketiga sisi dan `hitungLuas()` yang menerapkan rumus setengah alas kali tinggi secara spesifik.

Pembaruan pada kelas BidangDatarApps.java secara sistematis menunjukkan bagaimana sistem dapat diperluas untuk

mengakomodasi fitur baru dengan gangguan minimal terhadap struktur kode yang sudah ada. Penambahan menu keempat untuk opsi “Segitiga Siku” memungkinkan pengguna berinteraksi secara dinamis dengan memasukkan nilai alas dan tinggi yang kemudian diproses menjadi instansiasi objek Segitiga yang baru. Sebagai contoh, pemberian input alas senilai 16 dan tinggi 12 secara otomatis menghasilkan perhitungan keliling sebesar 48.0 serta luas sebesar 96.0. Keberhasilan integrasi ini menegaskan bahwa sistem yang menggunakan prinsip SOLID sangat efektif untuk pengembangan perangkat lunak masa kini, karena mempermudah proses skalabilitas tanpa merusak integritas alur kerja aplikasi.